TCP Echo server listening to port 5000
Client connected: /127.0.0.1:50303
Received: hi
Received: hi i am kamalesh

Type message to send (type 'bye' to quit)
Server: Echo hi
hi i am kamalesh
Server: Echo: i am kamalesh

Program:
Import j

Exp 12a
17/9

# END TO END COMMUNICATION AT TRANSPORT LAYER

## a) Echo Client - Server Using TCP

Aim:

To implement an echo client server communication program using TCP socket programming, where the server receives a message from client.

Procedure:

1) Create a TCP socket at server
2) Blind the server socket to IP
3) Make the server listen for incomming client
4) Accept the client connection
5) Create a TCP socket
6) Connect the client
7) Client sends message to server
8) Client receives a message
9) Close the socket after communication

Result:

Thus the TCP echo client-server communication was successfully implemented and message sent by client was received back.

UDP echo server listening to Port

Received from 127.0.0.1.6582 21 -> hi

Received from 127.0.0.1.6582 21 -> hi how are you

Type message to send

hi

Server: Echo: hi

hi how are you

Server: Echo: hi how are you

UDP socket is

Aim:

To implement client server communication using UDP sockets.

Procedure:

* Create a UDP socket at server
* Bind the server socket to an IP address and port number.
* Create a UDP socket at client
* Server receives message using recvfrom
* Server processes the received message
* Server sends a response
* Display the received message.
* Close the socket after communication

Result:

Thus UDP socket communication using socket programming was successfully implemented.

Enter your Name:
Kamalesh

Welcome, Kamalesh !

Sa

| | | | |
|---|---|---|---|
| TCP | 0.0.0.0.5000 | 0.0.0.0 | Listening |
| TCP | 127.0.0.1.5000 | 127.0.0.1 | Established |
| TCP | 127.0.0.1.50503 | 127.0.0.1 | Established |
| TCP | [::]:5000 | [::]:0 | listening |

EXP IV
17/9

# CHAT PROGRAM USING SOCKET PROGRAMMING

Aim :

To implement a chat application using socket programming that allows a client and server.

Procedure :

* Create a socket at the server
* Bind the socket to an IP address
* Make server listen for connections.
* Accept client connection.
* Create a socket at client.
* connect the client
* Server enters a reply
* Client receives and displays the reply.
* Repeat the send and receive process
* Terminate & close the sockets.

Result :

Thus a client-server chat program using socket programming was successfully implemented.